

Лабораторная работа № 10

Transfer Learning для классификации видов птиц

Цель работы: Реализовать систему классификации 200 видов птиц с использованием transfer learning на основе предобученной на ImageNet модели ResNet50.

Краткое описание работы. Пояснения к коду:

Внимание! Работа всего кода занимает 20-25 минут.

Этап 1. Подготовка данных.

Код подготавливает рабочую среду для запуска нейросетевой задачи и поворачивает создает папки для хранения датасетов. **Обратите внимание на проверку работы GPU, если работает CPU, смените среду выполнения T4.**

Этап 2. Загрузка датасета.

Код скачивает и распаковывает датасет с изображениями 200 видов птиц (CUB-200-2011).

Этап 3. Загрузка датасета.

Код подготавливает данные для обучения нейросети: преобразует изображения, создает удобную структуру папок и загрузчики данных. Выполняемые операции включают:

1. Реструктуризация исходного датасета. Исходный датасет CUB-200-2011 поставляется в формате, не совместимом с загрузчиком ImageFolder фреймворка PyTorch. Изображения распределяются по директориям классов с разделением на обучающую и валидационную выборки.
2. Аугментация обучающей выборки. Для обучающих данных применяются стохастические геометрические и колориметрические преобразования (повороты, изменение яркости, контраста и насыщенности).
3. Формирование батчей с использованием DataLoader. Создает загрузчики, которые будут подавать данные в нейросеть маленькими порциями (батчами) по 32 фото.

Этап 4. Визуализация данных.

Код показывает примеры изображений из датасета, чтобы убедиться, что данные загружены корректно (берет первые 6 картинок из датасета, "возвращает" им нормальные цвета (которые были изменены для обучения), подписывает название вида птицы и выводит на экран).

Этап 5. Transfer Learning.

Код реализует перенос обучения (transfer learning) с использованием предобученной модели ResNet50. Загрузка предобученной модели, заморозка всех слоёв, определение количества классов, замена последнего слоя (оригинальный классификатор ResNet50 на новый, адаптированный под 200 классов.), размораживание последнего блока, размораживание нового классификатора.

Этап 6. Настройка обучения.

Конфигурируются гиперпараметры и компоненты процесса обучения. Функция потерь – комбинация логарифмической функции softmax и отрицательного логарифма правдоподобия. Оптимизатор (Adam) сконфигурирован с дифференцированными скоростями обучения для различных групп параметров. Планировщик скорости обучения реализует адаптивное уменьшение темпа обучения при отсутствии снижения значения функции потерь на валидационной выборке в течение заданного числа эпох, уменьшая его вдвое (factor=0.5).

Этап 7. Функции обучения.

Функция `train_epoch` реализует одну эпоху обучения, выполняя последовательность операций: прямой проход (`forward pass`), вычисление функции потерь, обратное распространение ошибки (`backward pass`) и обновление параметров оптимизатором. В процессе вычисляются среднее значение функции потерь и точность классификации на обучающей выборке.

Функция `evaluate` производит оценку модели на валидационных данных без вычисления градиентов, что позволяет снизить потребление памяти и ускорить вычисления. Модель переводится в режим оценки (`model.eval()`), что деактивирует слои регуляризации (`Dropout`).

Функция `train_model` координирует полный цикл обучения: последовательный вызов `train_epoch` и `evaluate` для каждой эпохи, обновление планировщика скорости обучения, сохранение наилучшей модели на основе максимума валидационной точности, а также накопление истории метрик для последующего анализа.

Этап 8. Обучение модели.

Запускается процесс обучения на заданном количестве эпох с использованием ранее сконфигурированных компонентов: модели, загрузчиков данных, функции потерь, оптимизатора и планировщика скорости обучения. Результаты обучения аккумулируются в словаре `history`.

Этап 9. Визуализация результатов.

Строятся графики динамики функции потерь (`loss`) и точности (`accuracy`) для обучающей и валидационной выборок в зависимости от номера эпохи. Графики сохраняются в файл и отображаются. Выводятся итоговая и наилучшая точность на валидации.

Этап 10. Анализ ошибок классификации

Строится матрица ошибок (`confusion matrix`) размера `num_classes × num_classes`, где элемент `(i, j)` отражает количество объектов истинного класса `i`, классифицированных как класс `j`. На основе матрицы выделяются наиболее частые ошибочные пары классов (`top-N` ошибок).

Этап 11. Тестирование на новых изображениях.

Реализован интерактивный цикл, позволяющий пользователю загружать произвольные изображения птиц. Для каждого изображения выполняется предобработка (масштабирование, кропирование, нормализация), прямой проход через модель, вычисление вероятностей классов с помощью функции `softmax` и вывод топ-5 наиболее вероятных классов с соответствующими значениями уверенности. Цикл продолжается до получения отрицательного ответа пользователя.

Этап 12. Сохранение модели.

Выполняется сериализация обученной модели с сохранением в файл (`bird_classifier_complete.pth`) словаря, содержащего: состояния весов модели (`state_dict`), отображение индексов на названия классов, количество классов и достигнутую точность на валидации. Такая структура позволяет в дальнейшем загрузить модель для инференса без необходимости повторного определения архитектуры.

Ход работы

- 1. Вспомните, в чем заключается метод переноса обучения (`transfer learning`).**
- 2. Запустите вспомогательный код, приведенный в работе. Разберитесь в работе кода. Поясните подробно каждый из 12 этапов.**

3. Выполните один из экспериментов:

а) Измените количество эпох на 5 и 30. Сравните результаты.

б) Измените стратегию заморозки: разморозьте также layer3. Сравните точность и время обучения.

в) Замените бэкбон ResNet50 на ResNet18 или ResNet101. Сравните количество параметров и точность

г) Измените размер батча на 16 или 64. Опишите влияние на обучение

```
# =====
# ЧАСТЬ 1: ПОДГОТОВКА ОКРУЖЕНИЯ
# =====

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import numpy as np
import time
import os
import requests
from tqdm import tqdm

# Проверка GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется устройство: {device}")

# Создание папки для данных
os.makedirs('data', exist_ok=True)
os.makedirs('models', exist_ok=True)

# =====
# ЧАСТЬ 2: ЗАГРУЗКА ДАТАСЕТА CUB-200-2011
# =====

print("\n" + "="*60)
print("ЗАГРУЗКА ДАТАСЕТА CUB-200-2011 (200 видов птиц)")
print("="*60)

# Скачивание датасета (если нет локально)
if not os.path.exists('data/CUB_200_2011'):
    print("Скачивание датасета CUB-200-2011...")
    !wget -q https://data.caltech.edu/records/65de6-vp158/files/CUB_200_2011.tgz
    !tar -xzf CUB_200_2011.tgz -C data/
    !rm CUB_200_2011.tgz
    print("Датасет загружен и распакован!")

# =====
# ЧАСТЬ 3: ПРЕДОВАБОТКА ДАННЫХ
# =====

print("\n" + "="*60)
```

```

print("ПРЕДОБРАБОТКА ДАННЫХ")
print("="*60)

# Трансформации для тренировочных данных (с аугментацией)
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

# Трансформации для валидации (без аугментации)
val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

# Загрузка датасета (структура папок должна быть создана)
# CUB-200-2011 имеет специальную структуру, поэтому используем ImageFolder
# Предварительно нужно создать папки по классам
print("Создание структуры папок для ImageFolder...")

# Путь к исходным данным
cub_path = 'data/CUB_200_2011'

# Чтение файла с информацией о классах
import pandas as pd

# Загрузка соответствия id -> название класса
classes_df = pd.read_csv(f'{cub_path}/classes.txt', sep=' ', header=None,
names=['id', 'name'])
class_names = classes_df['name'].values

# Загрузка разбиения на train/test
images_df = pd.read_csv(f'{cub_path}/images.txt', sep=' ', header=None,
names=['id', 'path'])
labels_df = pd.read_csv(f'{cub_path}/image_class_labels.txt', sep=' ', header=None,
names=['id', 'label'])
split_df = pd.read_csv(f'{cub_path}/train_test_split.txt', sep=' ', header=None,
names=['id', 'is_train'])

# Объединение данных
data_df = images_df.merge(labels_df, on='id').merge(split_df, on='id')

# Создание временной структуры папок для ImageFolder
temp_train_path = 'data/cub_train'
temp_val_path = 'data/cub_val'

```

```

os.makedirs(temp_train_path, exist_ok=True)
os.makedirs(temp_val_path, exist_ok=True)

# Создание папок для каждого класса
for class_id, class_name in enumerate(class_names, 1):
    os.makedirs(f'{temp_train_path}/{class_id}_{class_name}', exist_ok=True)
    os.makedirs(f'{temp_val_path}/{class_id}_{class_name}', exist_ok=True)

# Копирование файлов в соответствующие папки
from shutil import copy2

for idx, row in data_df.iterrows():
    src_path = f"{cub_path}/images/{row['path']}"
    class_folder = f"{row['label']}_{class_names[row['label']-1]}"

    if row['is_train'] == 1:
        dst_path = f"{temp_train_path}/{class_folder}/{row['path'].split('/')[-1]}"
    else:
        dst_path = f"{temp_val_path}/{class_folder}/{row['path'].split('/')[-1]}"

    copy2(src_path, dst_path)

print(f"Создано {len(class_names)} классов")
print(f"Тренировочных изображений: {len(data_df[data_df['is_train']==1])}")
print(f"Валидационных изображений: {len(data_df[data_df['is_train']==0])}")

# Создание датасетов
train_dataset = datasets.ImageFolder(temp_train_path, transform=train_transform)
val_dataset = datasets.ImageFolder(temp_val_path, transform=val_transform)

# Создание загрузчиков
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True,
num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_workers=2)

# =====
# ЧАСТЬ 4: ВИЗУАЛИЗАЦИЯ ДАННЫХ
# =====

def show_sample_images(dataset, num_images=6):
    """Показывает примеры изображений из датасета"""
    fig, axes = plt.subplots(1, num_images, figsize=(15, 3))

    for i in range(num_images):
        image, label = dataset[i]
        # Денормализация
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        image = image.numpy().transpose(1, 2, 0)
        image = std * image + mean
        image = np.clip(image, 0, 1)

        axes[i].imshow(image)
        # Показываем только первые 20 символов названия класса
        class_name = dataset.classes[label]

```

```

        short_name = class_name.split('.')[ -1 ][ :25 ] if '.' in class_name else
class_name[:25]
        axes[i].set_title(short_name, fontsize=8)
        axes[i].axis('off')

plt.tight_layout()
plt.show()

print("\nПримеры изображений птиц из датасета:")
show_sample_images(train_dataset)

# =====
# ЧАСТЬ 5: СОЗДАНИЕ МОДЕЛИ С TRANSFER LEARNING
# =====

print("\n" + "="*60)
print("СОЗДАНИЕ МОДЕЛИ С TRANSFER LEARNING")
print("="*60)

# Загрузка предобученной на ImageNet модели ResNet50
model = models.resnet50(pretrained=True)
print("✓ Загружена модель ResNet50, предобученная на ImageNet")

# Заморозка всех слоев
for param in model.parameters():
    param.requires_grad = False

# Количество классов в CUB-200-2011
num_classes = 200

# Замена последнего слоя (классификатора)
model.fc = nn.Sequential(
    nn.Dropout(0.2),
    nn.Linear(2048, 512),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(512, num_classes)
)

# Размораживаем последний блок для дообучения
for param in model.layer4.parameters():
    param.requires_grad = True

# Размораживаем новый классификатор
for param in model.fc.parameters():
    param.requires_grad = True

model = model.to(device)

# Подсчет параметров
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"Всего параметров: {total_params:,}")

```

```

print(f"Обучаемых параметров: {trainable_params:},\n"
      f"({100*trainable_params/total_params:.1f}%)")

# =====
# ЧАСТЬ 6: НАСТРОЙКА ОБУЧЕНИЯ
# =====

print("\n" + "="*60)
print("НАСТРОЙКА ОБУЧЕНИЯ")
print("="*60)

# Функция потерь
criterion = nn.CrossEntropyLoss()

# Оптимизатор с разными learning rate для разных слоев
optimizer = optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}
], weight_decay=1e-4)

# Планировщик learning rate
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=3
)

# =====
# ЧАСТЬ 7: ФУНКЦИИ ОБУЧЕНИЯ
# =====

def train_epoch(model, loader, criterion, optimizer, device):
    """Обучает модель одну эпоху"""
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    pbar = tqdm(loader, desc="Обучение")
    for images, labels in pbar:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    # Обновление прогресс-бара
    pbar.set_postfix({'loss': loss.item(), 'acc': 100*correct/total})

    return running_loss / len(loader), 100 * correct / total

```

```

def evaluate(model, loader, criterion, device):
    """Оценивает модель на валидационных данных"""
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in tqdm(loader, desc="Валидация"):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    return running_loss / len(loader), 100 * correct / total


def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
epochs, device):
    """Полный цикл обучения"""
    history = {
        'train_loss': [], 'train_acc': [],
        'val_loss': [], 'val_acc': [],
        'best_acc': 0
    }

    for epoch in range(epochs):
        print(f"\n{'='*50}")
        print(f"Эпоха {epoch+1}/{epochs}")
        print('='*50)

        start_time = time.time()

        # Обучение
        train_loss, train_acc = train_epoch(model, train_loader, criterion,
optimizer, device)

        # Валидация
        val_loss, val_acc = evaluate(model, val_loader, criterion, device)

        epoch_time = time.time() - start_time

        # Обновление планировщика
        scheduler.step(val_loss)

        # Сохранение истории
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

```



```

# Сохранение лучшей модели
if val_acc > history['best_acc']:
    history['best_acc'] = val_acc
    torch.save(model.state_dict(), 'models/best_bird_classifier.pth')
    print(f"    ✓ Сохранена лучшая модель (точность: {val_acc:.2f}%)")

print(f"\nРезультаты эпохи {epoch+1}:")
print(f"  Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
print(f"  Val Loss:    {val_loss:.4f}, Val Acc:    {val_acc:.2f}%")
print(f"  Время: {epoch_time:.1f} сек")

# Текущий learning rate
current_lr = optimizer.param_groups[0]['lr']
print(f"  Learning rate: {current_lr:.6f}")

return history

# =====
# ЧАСТЬ 8: ОБУЧЕНИЕ МОДЕЛИ
# =====

print("\n" + "="*60)
print("НАЧАЛО ОБУЧЕНИЯ")
print("="*60)

# Обучение (15 эпох для демонстрации)
history = train_model(
    model, train_loader, val_loader, criterion,
    optimizer, scheduler, epochs=15, device=device
)

# =====
# ЧАСТЬ 9: ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ
# =====

print("\n" + "="*60)
print("ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ")
print("="*60)

# Построение графиков
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# График потерь
axes[0].plot(history['train_loss'], label='Train Loss', marker='o')
axes[0].plot(history['val_loss'], label='Validation Loss', marker='s')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Динамика потерь (Loss)')
axes[0].legend()
axes[0].grid(True)

# График точности
axes[1].plot(history['train_acc'], label='Train Accuracy', marker='o')
axes[1].plot(history['val_acc'], label='Validation Accuracy', marker='s')
axes[1].set_xlabel('Epoch')

```

```

axes[1].set_ylabel('Accuracy (%)')
axes[1].set_title('Динамика точности (Accuracy)')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('training_history.png', dpi=150)
plt.show()

print(f"\nИтоговая точность на валидации: {history['val_acc'][-1]:.2f}%")
print(f"Лучшая точность на валидации: {history['best_acc']:.2f}%")

# =====
# ЧАСТЬ 10: МАТРИЦА ОШИБОК (TOP-N)
# =====

def show_top_errors(model, loader, device, class_names, top_n=5):
    """Показывает топ-N самых частых ошибок классификации"""
    model.eval()

    # Матрица ошибок (confusion matrix)
    conf_matrix = torch.zeros(num_classes, num_classes)

    with torch.no_grad():
        for images, labels in tqdm(loader, desc="Анализ ошибок"):
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            for t, p in zip(labels, predicted):
                conf_matrix[t, p] += 1

    # Поиск самых частых ошибок
    errors = []
    for i in range(num_classes):
        for j in range(num_classes):
            if i != j and conf_matrix[i, j] > 0:
                errors.append((i, j, conf_matrix[i, j].item()))

    errors.sort(key=lambda x: x[2], reverse=True)

    print(f"\nТоп-{top_n} самых частых ошибок:")
    print("-" * 60)
    for i, (true_idx, pred_idx, count) in enumerate(errors[:top_n]):
        true_name = class_names[true_idx].split('.')[-1][:40]
        pred_name = class_names[pred_idx].split('.')[-1][:40]
        print(f"{i+1}. {true_name} → {pred_name} (ошибка: {int(count)})")

    # Показываем топ ошибок
    show_top_errors(model, val_loader, device, train_dataset.classes, top_n=10)

# =====
# ЧАСТЬ 11: ТЕСТИРОВАНИЕ НА НОВЫХ ИЗОБРАЖЕНИЯХ
# =====

```

```

def predict_bird_species(image_path, model, device, class_names):
    """Предсказывает вид птицы для загруженного изображения"""
    from PIL import Image

    transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406],
                               std=[0.229, 0.224, 0.225])
    ])

    image = Image.open(image_path).convert('RGB')
    input_tensor = transform(image).unsqueeze(0).to(device)

    model.eval()
    with torch.no_grad():
        output = model(input_tensor)
        probabilities = torch.softmax(output, dim=1)
        top5_probs, top5_indices = torch.topk(probabilities, 5)

    # Показ результатов
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(image)
    plt.axis('off')
    plt.title('Загруженное изображение')

    plt.subplot(1, 2, 2)
    top5_names = [class_names[idx].split('.')[0] for idx in top5_indices[0]]
    plt.barh(range(5, 0, -1), top5_probs[0].cpu().numpy()[::-1])
    plt.yticks(range(1, 6), top5_names[::-1])
    plt.xlabel('Вероятность')
    plt.title('Топ-5 предсказаний')

    plt.tight_layout()
    plt.show()

    return top5_indices[0][0].item(), top5_probs[0][0].item()

# Циклическое тестирование (простой вариант)
print("\n" + "="*60)
print("ТЕСТИРОВАНИЕ НА НОВЫХ ИЗОБРАЖЕНИЯХ")
print("="*60)
print("Модель готова к распознаванию видов птиц")
print("Для завершения работы введите 'нет' или 'stop'")
print("-" * 60)

from google.colab import files

while True:
    print("\n Загрузите изображение птицы (или нажмите 'Отмена' для выхода)...")

```

```

# Загрузка изображения
uploaded = files.upload()

# Если пользователь не загрузил файл, завершаем
if not uploaded:
    print(" Изображение не загружено. Завершение работы.")
    break

# Анализ каждого загруженного файла
for filename in uploaded.keys():
    print(f"\n Анализ изображения: {filename}")

    try:
        pred_class, confidence = predict_bird_species(
            filename, model, device, train_dataset.classes
        )
        class_name = train_dataset.classes[pred_class].split('.')[0]

        print(f"\n{'='*50}")
        print(f" РЕЗУЛЬТАТ РАСПОЗНАВАНИЯ")
        print(f"\n{'='*50}")
        print(f" Предсказанный вид: {class_name}")
        print(f" Уверенность: {confidence:.2%}")
        print(f"\n{'='*50}")

    except Exception as e:
        print(f" Ошибка при анализе изображения {filename}: {e}")

# Вопрос о продолжении
while True:
    user_input = input("\n☞ Хотите проанализировать еще одно изображение?
(да/нет): ").strip().lower()
    if user_input in ['нет', 'no', 'н', 'н', 'stop', 'выход', 'exit']:
        print("\n Завершение работы. Спасибо за использование!")
        break
    elif user_input in ['да', 'yes', 'д', 'у']:
        print(" Продолжаем...")
        break
    else:
        print(" Пожалуйста, введите 'да' или 'нет'")

if user_input in ['нет', 'no', 'н', 'н', 'stop', 'выход', 'exit']:
    break

print("\n" + "="*60)
print(" ТЕСТИРОВАНИЕ ЗАВЕРШЕНО")
print("="*60)

# =====
# ЧАСТЬ 12: СОХРАНЕНИЕ МОДЕЛИ
# =====

# Сохранение полной модели
torch.save({
    'model_state_dict': model.state_dict(),

```

```
    'class_names': train_dataset.classes,
    'num_classes': num_classes,
    'val_accuracy': history['best_acc']
}, 'models/bird_classifier_complete.pth')

print("\n" + "="*60)
print("ЗАВЕРШЕНИЕ РАБОТЫ")
print("="*60)
print("Модель сохранена в папку 'models/'")
print(f"Лучшая точность на валидации: {history['best_acc']:.2f}%")
```

При сбоях загрузки датасета подгрузите код перезагрузки:

```
# Полная перезагрузка датасета
import os
import shutil

# Удаляем старую папку, если есть
if os.path.exists('data/CUB_200_2011'):
    shutil.rmtree('data/CUB_200_2011')

# Скачиваем и распаковываем заново
!wget -q https://data.caltech.edu/records/65de6-vp158/files/CUB_200_2011.tgz
!mkdir -p data
!tar -xzf CUB_200_2011.tgz -C data/
!rm CUB_200_2011.tgz

# Проверяем
!ls data/CUB_200_2011/ | head -10
```